

Computer Engineering Project Course

ASSIGNMENT 2

Module Interface & Header Design

Smart Agriculture / Weed Elimination Robot

Autonomous On-Device Plant Inspection and Precision Actuation System

CSE396 — Group 9

Name	Student ID	Assigned Module
Berat Yılmaz, Mehmet Efe Hırkalı	2201004004069, 230104004071	Pathing
Mustafa Başaran, Salih Cengiz	220104004084, 220104004007	Image Processing
Umut Akman, Bekir Göktepe	250104004997, 220104004018	VLM
Muhammed Paşa, Fatih Mehmet Serenli	220104004930, 220104004012	Decision Motor
Umut Akman, Muhammed Paşa	250104004997, 220104004930	GUI

Submission Date: 29.03.2026

3.2. Module-Student Matching Table

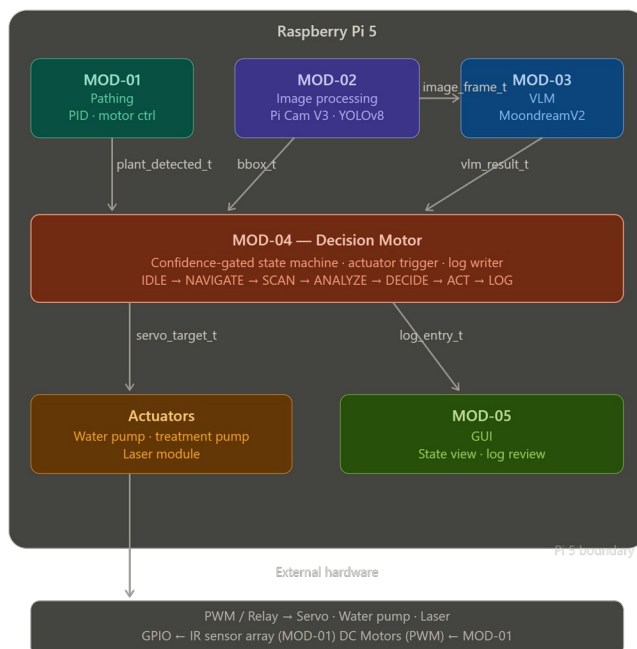
The table below shows which student is responsible for each module.

Module ID	Module Name	Responsible Student(s)	Brief Role
MOD-01	Pathing	Berat Yılmaz, Mehmet Efe Hırkalı	PID line-following with IR array, plant marker detection, motor control
MOD-02	Image Processing	Mustafa Başaran, Salih Cengiz	Pi Camera V3 capture, OpenCV preprocessing, YOLOv8-nano bounding box detection
MOD-03	VLM	Umut Akman, Bekir Göktepe	MoondreamV2 (INT4) inference, prompt engineering, JSON output parsing
MOD-04	Decision Motor	Muhammed Paşa, Fatih Mehmet Serenli	Confidence-gated decision logic, servo targeting, actuator control (pump/laser)
MOD-05	GUI	Umut Akman, Muhammed Paşa	Real-time monitoring, camera feed display, decision log review

3.3. System / Module Overview

Block Diagram

The diagram below shows the five software modules running on the Raspberry Pi 5 and the hardware peripherals they interface with. Arrows indicate the direction of data flow between modules.



Module Descriptions

MOD-01 — Pathing: The Pathing module keeps the robot on the black tape line using a 5-sensor IR array and a PID controller running at 100 Hz. It continuously reads sensor values, computes a weighted error, and adjusts left and right DC motor speeds via the L298N driver to correct the robot's heading. When all five IR sensors detect a cross-tape plant marker simultaneously, the module stops the robot and fires a callback to MOD-04 (Decision Motor) to initiate the inspection pipeline. After MOD-04 completes actuation or skips, it calls `pathing_resume()` to restart navigation; state transitions are also broadcast to MOD-05 (GUI) in real time.

MOD-02 — Image Processing: Manages the full camera pipeline from raw frame capture to structured detection output. Uses `picamera2` to acquire frames from the Pi Camera Module V3, applies OpenCV preprocessing (resize, color-space conversion, contrast normalization), and runs YOLOv8-nano inference to locate plant regions within the frame. Outputs a bounding box struct (pixel coordinates + detection confidence) consumed by the Decision Motor for servo targeting, and forwards the cropped ROI image to the VLM module for semantic analysis. Also streams annotated frames to the GUI for live monitoring.

MOD-03 — VLM (Vision-Language Model): Performs multi-modal plant analysis by accepting cropped plant images and environmental sensor context, then generating structured JSON output with classification, confidence scoring, and action recommendations. Receives a cropped ROI (region of interest) from the Image Processing. Uses MoondreamV2 (INT4 quantized) running on the Pi CPU to process the image against a structured prompt that requests JSON output containing: plant status {healthy, diseased, weed}, confidence score (0.0–1.0), diagnosis text, recommended action {skip, spray, laser}, and severity level {low, medium, high}. The module is designed for prompt-driven flexibility — behavior is controlled entirely through natural language prompts without model retraining. Returns the structured result to the Decision Motor for confidence-gated action logic.

MOD-04 — Decision Motor: Implements the confidence-gated decision logic and controls physical actuation. Receives the VLM result and bounding box coordinates, classifies confidence into three tiers (high ≥ 0.7 act immediately, mid 0.5–0.7 re-evaluate once, low < 0.5 skip), computes servo pan-tilt angles via pixel-to-angle calibration, and triggers the water pump (spray for healthy/diseased) or laser pointer (for weeds). Logs all decisions with full context for the GUI module.

MOD-05 — GUI: Provides the operator-facing dashboard and field report interface for real-time monitoring and post-run review. This module runs as a lightweight HTTP server on the Raspberry Pi and serves a browser-based dashboard with an MJPEG live camera stream, WebSocket-based robot status updates, real-time VLM result display, mission control endpoints (start / pause / resume / abort), and SQLite-backed persistence for plant analysis records. The dashboard is optional and does not participate in the core autonomy loop; the robot continues operating without WiFi or an active client connection.

Overall System Architecture

The system runs entirely on a Raspberry Pi 5 with no cloud dependency. All five software modules execute as components of a single Python/C application on Raspbian OS. Hardware peripherals (IR sensors, motors, camera, servos, pump, laser) are accessed via GPIO and the Pi Camera interface. The AI pipeline (YOLOv8-nano + MoondreamV2) runs on the Pi CPU using INT8/INT4 quantisation for edge-feasible inference times.

3.4. Hardware & Platform Summary

Target Hardware

Component	Model / Specification
-----------	-----------------------

Main Computer	Raspberry Pi 5
Camera	Raspberry Pi Camera Module
IR Sensor Array	TCRT5000 × 5 (digital output, 3.3 V compatible)
Motors	DC Motor × 4 (4WD chassis kit), L298N dual H-bridge driver
Servo Pan-Tilt	MG996R × 2 (pan + tilt), metal-gear servo, 50 Hz PWM
Water Pump	12 V DC mini submersible pump, 5 V relay module, 250 ml reservoir
Laser Pointer	5 mW red laser module (650 nm), 5 V relay-controlled
Power	LiPo 11.1 V 3S 3000 mAh; 5 V / 5 A step-down regulator (Pi + logic); 12 V regulator (pump)
Chassis	4WD robot chassis kit (custom assembled) — acrylic frame, 4 × TT DC motors, 65 mm wheels

Communication Buses

Bus / Interface	Used Between	Notes
GPIO (digital)	Pi ↔ IR sensor array	5 digital input pins, polled in PID loop
GPIO (PWM)	Pi ↔ DC motors (via L298N)	2 PWM channels for speed, 4 digital for direction
GPIO (PWM)	Pi ↔ SG90 servos	2 PWM channels (pan + tilt), 50 Hz
GPIO (digital)	Pi ↔ Relay modules	2 digital output pins (pump relay, laser relay)
CSI / libcamera	Pi ↔ Pi Camera V3	MIPI CSI-2 interface, accessed via picamera2 library
Internal (memory)	Between software modules	All modules share data via function calls and shared structs in-process

Clock / Timing Constraints

Parameter	Value
PID control loop rate	~100 Hz (10 ms period)
YOLOv8-nano inference	~1–3 seconds per frame on Pi 5 CPU
MoondreamV2 inference	~5–15 seconds per frame on Pi 5 CPU
Servo response time	~100–200 ms to reach target angle

Pump activation duration	2 seconds (configurable)
Laser activation duration	3 seconds (configurable)
Target per-plant total time	≤ 30 seconds

External Libraries / SDKs

Library	Purpose
OpenCV (cv2)	Image capture, preprocessing, display
Ultralytics (YOLOv8)	Object detection inference
llama.cpp / llama-cpp-python	MoondreamV2 VLM inference (INT4)
picamera2	Pi Camera V3 interface
RPi.GPIO / gpiozero	GPIO control for sensors, motors, servos, relays
NumPy	Image array manipulation
cpp-httplib / Crow	Browser-based GUI dashboard served over HTTP/WebSocket

3.5. Inter-Module Communication Description

For every pair of modules that exchange data, the table below describes the communication in detail. Detailed prose descriptions follow the table.

Communication Table

From	To	Data / Signal	Type	Direction	Notes / Frequency
MOD-01 Pathing	MOD-04 Decision Motor	Plant detected signal	Callback	01 → 04	Event-driven: once per plant marker
MOD-04 Decision Motor	MOD-01 Pathing	Action complete signal	Function call	04 → 01	Once per plant, after actuation or skip
MOD-02 Image Proc.	MOD-03 VLM	Cropped ROI image	image_frame_t	02 → 03	Once per plant stop (or twice on re-eval)
MOD-02 Image Proc.	MOD-04 Decision Motor	Bounding box coordinates	bbox_t	02 → 04	Once per detection, for servo targeting
MOD-03 VLM	MOD-04 Decision Motor	Analysis result	vlm_result_t	03 → 04	Once per VLM inference
MOD-04 Decision	MOD-02 Image	Re-capture	Function call	04 → 02	On mid-confidence (0.5–

Motor	Proc.	request			0.7), max 1 retry
MOD-04 Decision Motor	MOD-03 VLM	Re-analysis request	Function call	04 → 03	On mid-confidence, with new ROI
MOD-04 Decision Motor	MOD-05 GUI	Decision log entry	log_entry_t	04 → 05	Once per plant, via gui_append_log()
MOD-04 Decision Motor	MOD-05 GUI	Robot state update	robot_state_t	04 → 05	On every state transition
MOD-02 Image Proc.	MOD-05 GUI	Camera feed + overlay	image_frame_t + bbox_t	02 → 05	Per frame, via gui_update_feed()
MOD-03 VLM	MOD-05 GUI	VLM result display	vlm_result_t	03 → 05	Per inference, via gui_show_vlm_result()
MOD-01 Pathing	MOD-05 GUI	Navigation state	robot_state_t	01 → 05	On state transitions

Detailed Communication Descriptions

MOD-01 (Pathing) → MOD-04 (Decision Motor): Plant Detection Signal

When the Pathing module's IR sensors detect a plant marker, the robot stops and fires a registered callback (pathing_plant_detected_cb_t). This triggers the pipeline: Image Processing captures a frame, VLM analyses the ROI, and Decision Motor evaluates the result. The callback mechanism allows the Decision Motor (or a central orchestrator) to initiate the scan-analyse-decide sequence without tight coupling to the Pathing module's internal loop.

Data exchanged: No data payload — the callback signals “robot is stopped at a plant, ready for inspection.”

MOD-04 (Decision Motor) → MOD-01 (Pathing): Resume Navigation

After the Decision Motor completes its action (or decides to skip), it calls pathing_resume() to signal that the robot should continue navigating to the next plant marker. This ensures the robot does not move while an actuator is active.

Data exchanged: No data payload — function call signals “action complete, safe to move.”

MOD-02 (Image Processing) → MOD-04 (Decision Motor): Bounding Box

The Image Processing module provides a bbox_t struct containing the pixel coordinates (x, y, width, height) and YOLO detection confidence of the detected plant region. The Decision Motor uses the bounding box center to compute servo pan-tilt angles via dm_compute_servo_target(), enabling the spray or laser to be aimed at the detected plant.

Data exchanged: bbox_t struct — pixel coordinates and detection confidence.

MOD-03 (VLM) → MOD-04 (Decision Motor): Analysis Result

The VLM module provides a `vlm_result_t` struct with the parsed JSON output from MoondreamV2: plant status (healthy/diseased/weed), confidence score (0.0–1.0), text diagnosis, recommended action (skip/spray/laser), and severity level. The Decision Motor applies its confidence thresholds to this result to determine whether to act, re-evaluate, or skip.

Data exchanged: `vlm_result_t` struct — classification, confidence, diagnosis, action, severity, parse success flag.

MOD-02 (Image Processing) → MOD-03 (VLM): Image ROI

When the Image Processing module completes YOLO detection, it crops the plant region from the full frame and passes it to the VLM module via `vlm_analyze_plant()`. The input includes the cropped image data (RGB pixel buffer) and image dimensions. The VLM module constructs a multi-modal prompt incorporating the visual data, then invokes MoondreamV2 to generate a structured JSON response.

Data exchanged: `vlm_image_t`: RGB image buffer (`uint8_t` array), width, height, stride

MOD-04 (Decision Motor) → MOD-02/MOD-03: Re-Evaluation Request

When the VLM confidence falls in the mid range (0.5–0.7), the Decision Motor fires a `DM_EVENT_REEVAL_REQUESTED` callback and directly calls `imgproc_capture_and_detect()` followed by `vlm_analyze()` to re-capture and re-analyse the plant. This happens at most once per plant (`DM_MAX_REEVALUATIONS = 1`). On the second evaluation, the result is accepted regardless of confidence.

Data exchanged: Uses the standard `imgproc_result_t` and `vlm_result_t` return types from the respective modules.

MOD-04 (Decision Motor) → MOD-05 (GUI): Decision Logs & State

After every plant evaluation, the Decision Motor populates a `log_entry_t` struct (plant ID, timestamp, bounding box, VLM result, action taken, evaluation count, saved image path) and passes it to `gui_append_log()`. State transitions (DECIDE, ACT, LOG) are broadcast via `gui_update_state()` so the GUI can display the current robot state in real time.

Data exchanged: `log_entry_t` struct and `robot_state_t` enum.

MOD-02 (Image Processing) → MOD-05 (GUI): Annotated Camera Feed

The Image Processing module forwards every captured and annotated frame to the GUI via `gui_update_feed()`. The frame passed is the full-resolution image with YOLO bounding boxes already drawn (green rectangle + confidence score label). This call is made once per detection cycle — approximately once per plant stop — rather than at a continuous video rate, since inference is the bottleneck. The GUI module is responsible for rendering the frame; Image Processing does not retain a reference after the call.

Data exchanged: `image_frame_t` (annotated full frame) + `bbox_t` (bounding box coordinates and confidence for overlay rendering).

3.6. Known Risks & Open Questions

- 1. Servo calibration accuracy:** The pixel-to-angle mapping assumes a linear field-of-view model. Pi Camera V3's wide-angle lens may introduce barrel distortion that degrades targeting accuracy at frame edges. May need a non-linear calibration lookup table.
- 2. VLM inference latency:** MoondreamV2 (INT4) inference on Pi 5 CPU may take 5–15 seconds per plant. Combined with re-evaluation on mid-confidence, worst-case per-plant time could exceed the 30-second target. May need to reduce image resolution or explore ONNX Runtime as an alternative to llama.cpp.
- 3. GPIO pin allocation:** Final GPIO pin assignments for sensors, motors, servos, and relay modules have not been finalized. Need to verify total pin count and ensure no conflicts with the CSI camera interface.
- 4. Power supply stability:** Running YOLO + VLM inference causes CPU load spikes. Combined with servo and pump actuation drawing from the same power rail, voltage drops could cause Pi brownouts. May need separate voltage regulators for logic vs. actuators.
- 5. Re-evaluation loop edge case:** If the VLM consistently returns mid-confidence (0.5–0.7) on certain plant types, the single re-evaluation may not resolve the ambiguity. Need a fallback policy — current design skips after 1 retry, but this should be monitored during testing.
- 6. Vision-Only Classification Insufficient Accuracy:** Without environmental sensor context (soil moisture, lighting), the VLM may produce lower confidence scores or misclassify plants, especially under variable lighting conditions in the parkour environment.
- 6. YOLOv8-nano false detections on non-plant objects:** The detector may trigger on background objects (track surface markings, cables, pots without plants) if the confidence threshold is set too low. Need to tune the YOLO confidence threshold during testing and potentially add a minimum bounding-box area filter to suppress small spurious detections.
- 7. Lighting variability and image quality:** The Pi Camera V3 auto-exposure may produce overexposed or underexposed frames under variable indoor lighting, degrading both YOLO detection accuracy and VLM analysis. Plan to fix camera exposure and white-balance settings at pipeline startup and test under the actual demo lighting conditions.
- 8. Security and access control not yet defined:** The public interface exposes HTTP routes, WebSocket connections, and mission-control callbacks, but the header does not specify authentication, authorization, or network restrictions. If the robot is ever connected to a shared lab or field network, unauthorized access to live data or mission commands becomes a risk.